

Министерство науки и высшего образования Российской
Федерации



Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 4 по дисциплине «Анализ алгоритмов»

Тема Параллельные вычисления на основе нативных потоков

Студент Пантелеев Василий Максимович

Группа ИУ7-52Б

Преподаватели Строганов Д. В. Волкова Л. Л.

Москва, 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Входные и выходные данные	3
2 Преобразование входных данных в выходные	4
3 Тестирование	4
4 Описание исследования	4
4.1 Технические характеристики	5
4.2 Результаты исследования	5
ЗАКЛЮЧЕНИЕ	6
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	7

ВВЕДЕНИЕ

Многопоточность — это метод параллельного выполнения, при котором несколько потоков обрабатывают задачи одновременно в рамках одного процесса, используя общую память и ресурсы. Она позволяет ускорить выполнение задач, которые могут выполняться параллельно, таких как обработка больших объемов данных и выполнение сложных вычислений. Благодаря многопоточности приложения могут эффективно задействовать все ядра процессора, что особенно важно для многопроцессорных систем [1].

Нативные потоки (Native Threads) — это потоки, управляемые непосредственно операционной системой. Использование нативных потоков позволяет создавать высокопроизводительные многопоточные приложения, но требует контроля доступа к общим ресурсам для предотвращения гонок данных.

Цель работы — получение навыка организации параллельных вычислений на основе нативных потоков. Для достижения поставленной цели необходимо выполнить следующие задачи:

- описать входные и выходные данные;
- реализовать два алгоритма для загрузки контента из html-страниц — последовательный и параллельный с использованием нативных потоков;
- протестировать разработанные алгоритмы;
- сравнить скорость выполнения программы в зависимости от количества используемых потоков.

1 Входные и выходные данные

Входными данными программы являются адрес главной страницы ресурса [2], а также количество потоков и максимальное количество страниц, с которых выгружаются данные. Выходными данными является директория, содержащая скачанные данные со страниц в формате, пригодном для дальнейшей обработки (html)

2 Преобразование входных данных в выходные

Потоки выполняют скачивание очередной страницы, сохранение результата в файл, а также поиск на странице ссылок на другие страницы с добавлением ссылок в структуру данных типа множество строк.

3 Тестирование

В таблице 3.1 представлены функциональные тесты для разработанного ПО. Все тесты пройдены успешно.

Таблица 3.1 – Функциональные тесты

Описание теста	Выходные данные	Ожидаемые выходные данные
Число страниц для загрузки ≤ 0	Сообщение об ошибке	Сообщение об ошибке
Число потоков ≤ 0	Сообщение об ошибке	Сообщение об ошибке
Некорректный url первой страницы	Сообщение об ошибке	Сообщение об ошибке
Работа в параллельном режиме, 4 потока	Директория <i>data</i> с 4 файлами	Директория <i>data</i> с 4 файлами
Работа в последовательном режиме, 1 поток	Директория <i>data</i> с 1 файлом	Директория <i>data</i> с 1 файлом

4 Описание исследования

В ходе исследования требуется сравнить скорость выполнения программы в зависимости от количества используемых потоков. Замеры проводились при обработке 100 страниц. Число дополнительных потоков изменялось с 1 до 16. По результатам исследования был построен график с использованием библиотеки *matplotlibpython* [3]

4.1 Технические характеристики

Технические характеристики используемого устройства:

- процессор: AMD Ryzen™ 7–5800H with Radeon™ Graphics × 16;
- ОС: 64-разрядная операционная система Ubuntu 24.04.1 LTS;
- оперативная память: 16,0 ГБ;
- число логических ядер – 16.

4.2 Результаты исследования

В таблице 4.1 приведено время выполнения программного обеспечения в секундах.

Таблица 4.1 – Сравнение скорости работы алгоритма от числа потоков

Число потоков	Время, секунды
1	18.42
2	11.87
4	6.07
8	3.37
12	2.52
16	2.32

На рисунке 4.1 показана зависимость времени работы от количества потоков с постоянным количеством обрабатываемых страниц.

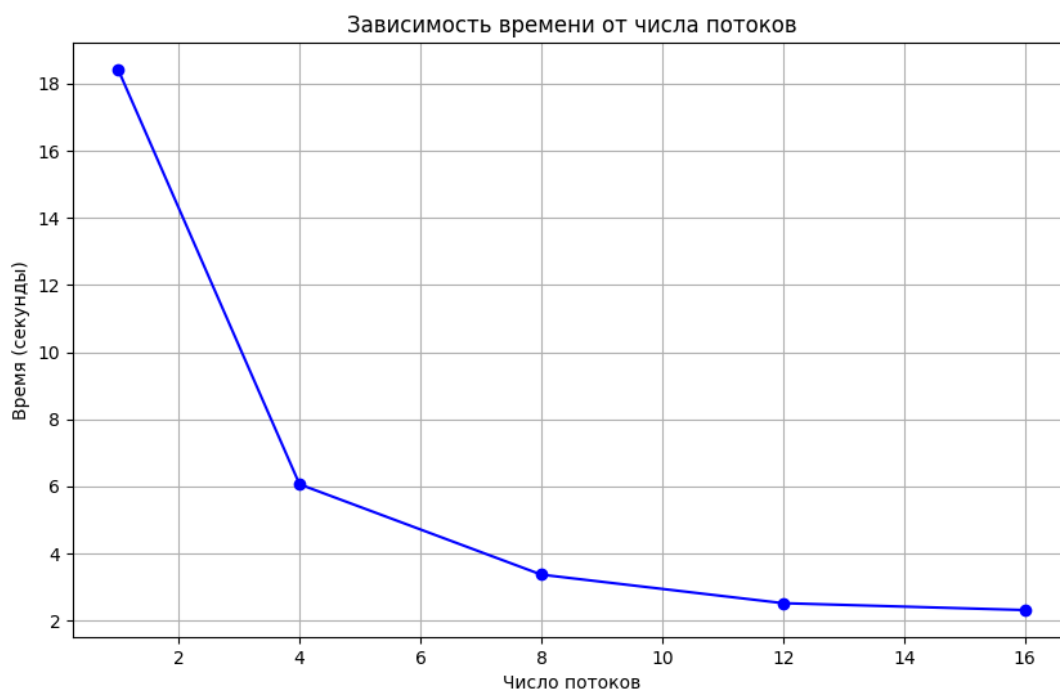


Рисунок 4.1 – Зависимость времени работы от количества потоков.

ЗАКЛЮЧЕНИЕ

В ходе исследования было установлено, что многопоточная реализация работает гораздо быстрее последовательной (примерно в 4,5 раза), скорость обработки страниц возрастает в 1.5 раза при увеличении числа потоков в 2 раза.

Цель работы была достигнута. Решены все поставленные задачи:

- описаны входные и выходные данные;
- реализованы 2 алгоритма обработки страниц: многопоточный и последовательный;
- разработанные алгоритмы протестированы;
- выполнено сравнение скорости выполнения программы в зависимости от количества используемых потоков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Уильямс Э. Параллельное программирование на C++ в действии. Практика разработки многопоточных программ [Электронный ресурс]. — 2012. — Режим доступа: <https://dodo.inm.ras.ru/konshin/НПС/bib/НПС-схх11-book.pdf> (дата обращения: 6.11.2024).
2. Кулинарные рецепты [Электронный ресурс]. — Режим доступа: <https://sytyikrolik.ru/> (дата обращения: 3.11.2024).
3. Библиотека matplotlib [Электронный ресурс]. — Режим доступа: <https://matplotlib.org/> (дата обращения: 6.11.2024).